# DOMAIN 4.1–4.5 // ATTACKS & EXPLOITS

COMPTIA PENTEST+ PT0-003 · PRIORITIZATION · NETWORK · AUTH · HOST · WEB

4.0 = 35% OF EXAM
THE ONE TO OVER-PREPARE

## HOW TO READ THIS GUIDE — THE SHAPE OF EVERY 4.X QUESTION

Domain 4 is "given a scenario, **perform**". Almost every question follows one shape: **here is output or a situation — pick the attack, the tool, or the next step.** Two habits win points: (1) identify the *artifact* you're pointed at (a hash file, a login form, an API, a running app), then the tool follows; (2) watch for the *reversible/irreversible* and *production* cues that make a "safer path" the correct answer.

> **THE FIVE CANONICAL TOOLS THAT RECUR**
> **Mimikatz** (cred dumping), **CrackMapExec** (spray/validate broadly), **Impacket** (AD/SMB glue), **Responder** (LLMNR/NBT-NS poisoning), **Scapy** (packet crafting). When a question is ambiguous and one of these fits, it's usually the intended answer.

## 4.1 PRIORITIZE & PREPARE ATTACKS — GIVEN A SCENARIO, ANALYZE OUTPUT

No tools — this is judgment. It decides *which* confirmed finding to attack and *how* to prepare. The metrics cluster is the highest-yield content in the objective.

### DESCRIPTORS & METRICS — KEEP THE FOUR STRAIGHT

| METRIC | WHAT IT IS | KEYWORD |
|---|---|---|
| **CVE** | Identifier for one *specific* vulnerability (CVE-2024-3094) | "which vulnerability" / a CVE string → identity, not score |
| **CWE** | Classifies the underlying *weakness type* (CWE-89 = SQLi) | "class of flaw", "root weakness category" |
| **CVSS** | Severity *score* 0–10 (base = intrinsic, context-free) | "how severe" |
| **EPSS** | Probability it's *exploited in the wild* | "how likely to be exploited", "prioritize by real-world risk" |

> EXAMPLE Two findings: CVE-A is CVSS 9.8 on an isolated lab host with no data; CVE-B is CVSS 6.5 on the internet-facing billing app with an EPSS of 0.87 and a public Metasploit module. ATTACK B FIRST — likelihood × business impact beats raw severity. This is the single most-tested 4.1 idea.

- **Target prioritization**
  High-value assets, end-of-life software, default configs, running services, vulnerable encryption, defensive capabilities.
  KEY "which target first" = value × exploitability × likely detection
- **Capability selection**
  Tool selection, exploit selection & customization (code analysis), documentation (attack path, low-level diagram, storyboard), dependencies, scope limitations, labeling sensitive systems.
  KEY "before running a public exploit" → review the code first (backdoors)

> **EXAM TELLS**
> "Prioritize these findings" → combine **CVSS + EPSS + business impact**, never severity alone. "Before you run this exploit from GitHub" → **review the source** for malicious payloads. "Production system + memory-corruption exploit" → authorize / schedule a window / pick a safer path (ties to 3.2 and 1.1 RoE). "Label sensitive systems" → mark them off-limits or handle-with-care before attacking.

## 4.2 NETWORK ATTACKS — GIVEN A SCENARIO, PERFORM

| ATTACK | WHAT IT DOES / EXAMPLE | KEYWORD & TOOL |
|---|---|---|
| **On-path (MITM)** | Intercept traffic between two parties — e.g. ARP-spoof a host to read its session | "intercept between", "position in the middle" → Ettercap, Responder, Wireshark |
| **LLMNR/NBT-NS poisoning** | Answer broadcast name lookups to capture NTLM hashes — e.g. victim mistypes a share, you reply | "broadcast name resolution", "captured NTLMv2" → **Responder** |
| **Relay attack** | Forward captured auth to a *different* host in real time — never cracks it | "forwarded / relayed the hash" → Responder + **ntlmrelayx**; SMB signing defeats it |
| **Default credentials** | Vendor admin/admin left unchanged | "factory default", "never changed" → CME, Hydra |
| **VLAN hopping** | Escape segmentation via switch spoofing or double-tagging | "reach another VLAN", "802.1Q double tag" |
| **Multihomed host** | Pivot through a host bridging two segments (2+ NICs) | "host with two interfaces", "bridges networks" |
| **Certificate services** | AD CS misconfig (ESC1–8) for privilege escalation | "AD CS", "vulnerable template" → Certipy, Impacket |
| **Packet crafting** | Build custom packets for precise probing/exploitation | "craft a custom packet" → **Scapy**, Nmap NSE |

> EXAMPLE You run Responder and capture an `NTLMv2` hash from a workstation that queried a nonexistent hostname. Two paths: CRACK IT OFFLINE with hashcat (you learn the password), or RELAY IT with ntlmrelayx to another host where that user is admin (you get access without ever knowing the password). If the target has SMB SIGNING ENFORCED, the relay fails and you must crack instead.

> **THE DISTINCTION THEY TEST: RELAY VS CRACK**
> A **relay** forwards live authentication to another target and never learns the password — defeated by SMB signing. **Cracking** (4.3) takes the captured hash offline to recover plaintext — unaffected by signing, defeated by a strong password. Same captured hash, two completely different follow-ups.

> **TOOL PATTERNS**
> Broad cred spray/validate across a subnet → **CME**. Craft/inject custom packets → **Scapy**. Poison name resolution → **Responder**. Relay the result → **Impacket ntlmrelayx**. Exploit a known service CVE → **Metasploit**.

## 4.3 AUTHENTICATION ATTACKS GIVEN A SCENARIO, PERFORM

| ATTACK | WHAT IT DOES / EXAMPLE | KEYWORD & TOOL |
|---|---|---|
| **Password spraying** | *One* password vs *many* users — e.g. `Winter2025!` against every account | "stays under lockout", "one password, many users" → CME, Hydra |
| **Credential stuffing** | Known breached *user:pass* pairs reused across services | "breach dump", "password reuse" → Hydra, Medusa |
| **Brute-force / dictionary** | Many guesses vs *one* account | trips lockout online → usually done offline |
| **Offline cracking** | Recover plaintext from a captured hash file | "you have the hashes" → **hashcat / John**, never Hydra |
| **Mask attack** | Brute-force with a known pattern ( `?u?l?l?l?d?d?d?d` ) | "known password structure" → hashcat `-a 3` |
| **Pass-the-hash** | Authenticate with the NTLM hash itself — no plaintext needed | "reuse the hash", "no cracking" → Mimikatz, CME |
| **Pass-the-ticket** | Reuse a stolen Kerberos ticket | "stolen TGT/TGS" → Rubeus, Mimikatz |
| **Kerberoasting** | Request a service ticket, crack it offline for the service-account password | "SPN", "service account", "crackable TGS" → Rubeus + hashcat |
| **AS-REP roasting** | Crack accounts with pre-auth disabled | "Kerberos pre-authentication disabled" → Rubeus |
| **MFA fatigue** | Spam push prompts until the user approves | "push bombing", "approved to stop the notifications" |
| **LDAP injection** | Manipulate an LDAP query via app input | `*)(uid=*` style input → Burp |
| **OIDC / SAML** | Manipulate federated tokens / assertions | "SSO", "signed assertion", "token" → Burp |

> EXAMPLE — SPRAY VS STUFF VS BRUTE 3,000 users, lockout at 5 tries in 30 min. SPRAY : try `Spring2025!` once against all 3,000 (1 attempt each, no lockout). STUFF : take a breach dump of `jdoe:hunter2` pairs and replay them (relies on reuse). BRUTE : hammer `jdoe` with 10,000 guesses (locks the account at try 5). This is why spraying and offline cracking dominate real engagements.

> EXAMPLE — KERBEROASTING Any domain user can request a service ticket (TGS) for an account with an SPN. The ticket is encrypted with the service account's password hash, so you crack it *offline* with hashcat — no lockout, no noise on the target. Service accounts often have weak, non-expiring passwords, which is what makes this so productive.

> ONLINE VS OFFLINE — THE MASTER SPLIT
>
> **Hydra / Medusa** = online, live service, trips lockout. **hashcat / John** = offline, captured hash file, no lockout. "You captured / dumped the hashes" is always the offline signal. **BloodHound** maps AD paths — it doesn't attack.

> PASS-THE-X FAMILY
>
> These reuse a credential *artifact* so no plaintext is ever needed — the reason MFA on the password doesn't stop them, and the reason pass-the-hash survives even strong passwords. Remediation leans on credential hygiene, tiering, and disabling NTLM, not on password policy.

## 4.4 HOST-BASED ATTACKS — GIVEN A SCENARIO, PERFORM

| ATTACK | WHAT IT DOES / EXAMPLE | KEYWORD & TOOL |
|---|---|---|
| **Privilege escalation** | Local user → SYSTEM/root via a misconfig | `"become SYSTEM/root"` → PowerSploit, Seatbelt, WinPEAS/LinPEAS |
| **Credential dumping** | Pull hashes/tickets from LSASS or SAM | `"dump LSASS"`, `"extract from memory"` → **Mimikatz** |
| **Unquoted service path** | Space in an unquoted path + writable dir → SYSTEM | `C:\Program Files\...` unquoted → drop `Program.exe` |
| **Weak service perms** | Reconfigure a service you can edit to run your binary | `"writable service"`, `"modifiable binPath"` |
| **DLL hijacking / library injection** | Drop a malicious DLL a process loads by search order | `"DLL search order"`, `"missing DLL"` |
| **Process hollowing / injection** | Run code inside a legitimate process to blend in | `"inject into a running process"`, `"hollow"` |
| **Kiosk / shell escape** | Break out of a restricted shell or kiosk UI | `"restricted shell"`, `"kiosk breakout"` |
| **Circumventing security tools** | Disable/bypass AV/EDR/AppLocker | `"bypass AppLocker"` → **LOLBins** (rundll32, msbuild) |
| **Remote execution (w/ creds)** | Run commands on a remote host you have creds for | `"execute remotely with valid creds"` → **PsExec, Evil-WinRM** |
| **AD CS abuse (local)** | Abuse certificate templates from the host | `"request a cert as another user"` → **Certify**, Rubeus |

EXAMPLE — UNQUOTED SERVICE PATH A service runs `C:\Program Files\My App\service.exe` unquoted. Windows tries `C:\Program.exe` first, then `C:\Program Files\My.exe`, and so on. If you can write to `C:\` or `C:\Program Files\`, drop a malicious `Program.exe` or `My.exe` and it runs as the service's account — usually SYSTEM — at next start.

EXAMPLE — LOLBINS FOR EVASION AppLocker blocks unknown executables, but `rundll32.exe` and `msbuild.exe` are Microsoft-signed and allow-listed. You execute your payload *through* them — rundll32 calls an exported DLL function, msbuild compiles and runs inline C# — and the allow-list never fires because the running binary is trusted.

TOOL PATTERNS

**Mimikatz** = credential dumping / pass-the-hash. **Rubeus** = Kerberos tickets. **Certify** = AD CS abuse. **PsExec / Evil-WinRM** = remote exec once you hold creds. **WinPEAS/LinPEAS, Seatbelt** = automated priv-esc enumeration. **LOLBins** = trusted-binary allow-list bypass.

## 4.5 WEB APPLICATION ATTACKS — GIVEN A SCENARIO, PERFORM

### INJECTION FAMILY

| ATTACK | WHAT IT DOES / EXAMPLE | KEYWORD & TOOL |
|---|---|---|
| **SQL injection** | Input alters a DB query — `' OR 1=1--` bypasses login; `UNION SELECT` exfiltrates | "database", "query", `' OR 1=1` → **sqlmap**, always |
| **Command injection** | Input reaches a shell — `; cat /etc/passwd` | "passed to the OS/shell", `;` `\|` `&&` |
| **XSS (reflected/stored/DOM)** | Inject script that runs in a victim's browser — steal cookies | `<script>`, "runs in the browser", "steal session" |
| **Server-side template injection** | Input evaluated by the template engine — `{{7*7}}` → 49 | `{{ }}` evaluates, "template engine" |

### INCLUSION, FORGERY & OBJECT ATTACKS

| ATTACK | WHAT IT DOES / EXAMPLE | KEYWORD & TOOL |
|---|---|---|
| **Directory traversal** | Escape the web root — `../../../etc/passwd` | `../` sequences, "read arbitrary files" |
| **LFI / RFI** | Include a local or remote file → often a web shell | "include", "?page=" parameter → RFI = remote URL |
| **SSRF** | Make the server fetch an attacker URL — hit `169.254.169.254` for cloud creds | "server makes the request", "internal/metadata" (bridges to 4.6) |
| **CSRF** | Forge a state-changing request from a logged-in victim | "forged request", "no anti-CSRF token" |
| **IDOR** | Change an object ID to access others' data — `?id=1002` | "change the ID", "access another user's record" |
| **Deserialization** | Untrusted serialized object → code execution | "serialized object", "pickle/ObjectInputStream" → ysoserial |
| **JWT manipulation** | `alg:none` or weak signing key forges a token | "JWT", `alg:none`, "weak secret" |
| **Session hijacking** | Steal/replay a session token | "stolen cookie", "session token" |

> EXAMPLE — SQLI LOGIN BYPASS A login runs `SELECT * FROM users WHERE user='$u' AND pass='$p'`. Entering `' OR '1'='1'--` as the username makes the query always true and comments out the password check — you're logged in as the first user. sqlmap automates finding and exploiting this, then can dump the whole database with `--dbs --dump`.

> EXAMPLE — SSRF TO CLOUD TAKEOVER An image-fetch feature lets you supply a URL. Point it at `http://169.254.169.254/latest/meta-data/iam/security-credentials/` and the server returns the instance's temporary AWS keys. This is why SSRF is the bridge between 4.5 and 4.6 — a web bug becomes full cloud access.

> TOOL PATTERNS
>
> **sqlmap** = SQLi, always. **WPScan** = WordPress. **Gobuster/DirBuster** = find hidden content (a first step, not the exploit). **Burp Suite / ZAP** = the generalist for CSRF, IDOR, JWT, XSS, SSRF, session work. **Postman** = API-specific. **ysoserial** = deserialization. **TruffleHog** = secrets in code.

> REMEDIATION BRIDGE (1.5)
>
> The whole injection family (SQLi, command, XSS, SSTI, LDAP) remediates to **sanitize input / parameterize queries** — a WAF is a compensating control, not the fix. IDOR → enforce object-level authorization. CSRF → anti-CSRF tokens. Deserialization → never deserialize untrusted data.